

Track the Cash: Project File and Folder Guide

Generated for understanding the working principle of every important project folder and tracked file.

1. What the project does

Track the Cash is a predictive cybercrime-response application. It combines complaint data, mule-account geography, ATM locations, complaint velocity, and a machine-learning model to estimate which ATMs may be used for cash withdrawal in the next 24 hours.

The system has two parts:

- Backend: FastAPI HTTP API, authentication, SQLAlchemy database access, prediction and alert logic.
- Frontend: React single-page dashboard with an LEA tactical map and an I4C/Admin intelligence view.

The data flow is:

```
ATM data + complaint data + mule accounts
-> database
-> feature calculation
-> ML risk score and spike rules
-> FastAPI endpoints
-> React dashboard, alerts, and CSV export
```

2. Start here: runtime flow

Local development

1. Install backend dependencies from `backend/requirements.txt`.
2. Run `backend/scripts/init_db.py` to create tables.
3. Run `backend/scripts/fetch_atms.py` to import or generate ATM locations.
4. Run `backend/scripts/generate.py --mode batch --days 30` to create synthetic complaints and mule accounts.
5. Run `backend/scripts/train_model.py` to train/save the model and create predictions.
6. Start FastAPI with Uvicorn on port 8000.
7. Install frontend dependencies and run Vite. The browser calls the backend through the Vite proxy.

Docker startup

`docker-compose.yml` starts PostgreSQL, builds the API image, and serves the frontend with Nginx. The API container runs `backend/entrypoint.sh`, which performs database initialization, ATM ingestion, synthetic data generation, model training, and then starts Uvicorn.

Typical user request

1. `frontend/src/pages/Login.jsx` sends credentials to `/auth/login`.
2. `backend/app/routers/auth.py` checks the user through `backend/app/auth.py` and returns a JWT plus role.
3. `frontend/src/context/AuthContext.jsx` keeps the token in memory and supplies the Bearer header.
4. `frontend/src/App.jsx` routes the user to `/lea` or `/admin` and `ProtectedRoute.jsx` checks the role.

5. `LeaView.jsx` requests `/predict` and `/alerts/feed`; `AdminView.jsx` requests predictions, velocity, metrics, simulations, and reports.
6. Backend routers query SQLAlchemy models through `database.py`, calculate or retrieve risk, and serialize results through `schemas.py`.
7. `RiskMap.jsx` displays ATM scores as colored Leaflet markers; `AlertFeed.jsx` displays spike alerts.

3. Repository folders

`/backend`

The Python backend application, database migration setup, data scripts, Docker definitions, and tests.

`/backend/app`

The importable FastAPI application package. It contains configuration, authentication, database setup, ORM models, schemas, ML code, services, and HTTP routers.

`/backend/app/ml`

Feature engineering, model training/inference, and complaint-velocity spike detection.

`/backend/app/routers`

Small FastAPI route modules. Each module owns a group of HTTP endpoints and calls database/services code rather than making the frontend implement business rules.

`/backend/app/services`

Reusable business adapters: risk explanations, entity-network construction, notification delivery, and ML model abstraction.

`/backend/alembic`

Database migration package. Migration files describe schema changes that can be applied consistently to PostgreSQL or another supported database.

`/backend/scripts`

Command-line preparation and data-pipeline scripts. These are run during setup or model refresh, not on every browser request.

`/backend/tests`

Pytest tests for database constraints, authentication, data generation, ML, predictions, heatmaps, alerts, analytics, reports, and simulation.

`/frontend`

The browser application and its production Nginx container.

`/frontend/src`

React entry point, routing/layout, global styles, auth context, reusable dashboard components, and role-specific pages.

`/data`

Cached/static data used by setup scripts, especially the ATM cache created from fetched or fallback locations.

`/docs`

Product, architecture handoff, research, and generated project guides. Markdown is the editable source; PDFs are rendered outputs.

`/scripts`

Repository-level helper scripts. These are separate from backend data-pipeline scripts and are intended for project/document support.

`/.scratch`

Local issue-tracker material. Feature-specific notes live below this directory and are not part of runtime application code.

`/.agents`

Agent and repository workflow resources used during development. They do not run as part of the deployed application.

4. Root files

`AGENTS.md`

Instructions for coding agents working on this repository. It defines issue-tracker conventions, triage labels, and where domain documentation belongs.

`README.md`

The main operator/developer overview. It explains the product mission, architecture, demo accounts, quick start, tests, Docker commands, and demo script.

`docker-compose.yml`

Defines the three-container deployment: PostgreSQL on host port 5433, FastAPI on port 8000, and the Nginx-served frontend on port 3000. It also defines persistent database/model volumes and health checks.

`.env.example`

Template for environment variables such as database URL, JWT settings, SMTP credentials, and model path. Copy values into a local `.env`; never commit real secrets.

`skills-lock.json`

Locks agent skill/development metadata. It affects development tooling, not application runtime.

`track_the_cash.db`

Local SQLite database file used by the default development configuration. It is runtime state, not source code.

`.gitignore`

Lists files Git should not track, such as caches, local environments, generated model artifacts, and temporary files.

`.DS_Store`

Operating-system metadata created by macOS. It has no application behavior and should remain ignored.

5. Backend application files

`backend/app/config.py`

Defines the `Settings` object loaded from defaults, `.env`, and environment variables. It controls database URL, JWT behavior, SMTP/Twilio settings, model path, spike threshold, and demo mode.

`backend/app/database.py`

Creates the SQLAlchemy engine, session factory, and declarative Base. `get_db()` yields one request-scoped database session and closes it afterward.

`backend/app/models.py`

Defines the database tables and relationships. Main entities are users, complaints, mule accounts, transactions, ATMs, risk history, predictions, alerts, investigations, entity links, notification preferences, watchlists, and audit logs. Check constraints protect valid role, status, severity, and risk values.

`backend/app/schemas.py`

Defines Pydantic request/response models. Routers use these schemas to validate incoming JSON and shape outgoing API responses without exposing arbitrary ORM fields.

`backend/app/auth.py`

Implements demo-user lookup, password hashing/verification, JWT creation and decoding, current-user dependency, and role dependencies such as `require_admin`, `require_roles`, and `require_authenticated`.

`backend/app/email_service.py`

Builds and sends alert emails through SMTP using configured settings. Spike detection and notification routes call it when automatic or manual notification is required.

`backend/app/main.py`

The FastAPI application entry point. It configures OpenAPI/Swagger and CORS, registers every router, creates SQLite tables for local development, and exposes `GET /health`.

6. Backend ML files

`backend/app/ml/features.py`

Builds one feature row per ATM from database data. It calculates seven-day district fraud density, six-hour complaint velocity, nearest mule centroid distance using the Haversine formula, district ATM count, and cross-state status.

`backend/app/ml/model.py`

Owns the `ATMDenseModel`. It trains XGBoost when available, otherwise uses Gradient Boosting, computes ROC-AUC and Precision@10, saves/loads a Joblib artifact, and runs predictions for all ATMs. Before a trained model is available, it uses a bounded heuristic fallback.

`backend/app/ml/spike_detector.py`

Contains the pure spike rule and the database scanner. A six-hour complaint count above twice the seven-day six-hour-window average becomes a WARNING, or CRITICAL when cross-state activity is present. It stores alerts and avoids duplicate email delivery within six hours.

7. Backend service files

`backend/app/services/ml_adapter.py`

Defines a common risk-model interface plus mock and trained-model adapters. It lets API code use a model without depending directly on one ML implementation.

`backend/app/services/risk_service.py`

Builds the evidence shown beside a prediction: complaint factors, cross-state activity, and spike metrics for an ATM.

`backend/app/services/network_service.py`

Traverses stored entity links and related complaints/transactions to return investigation nodes and edges for mule-account or other entity network views.

`backend/app/services/notifications.py`

Defines email and WhatsApp provider abstractions and a notification service. It records delivery outcomes and gives the API a single place to send alert notifications.

8. Backend router files

All route modules are imported and registered by `backend/app/main.py`.

`admin.py`

Admin-only user management, ATM import, and administrative statistics endpoints.

`alerts.py`

Creates or lists alerts, exposes the legacy feed, performs acknowledgement/escalation/resolution actions, returns delivery history, and sends notifications. It also connects to spike detection and manual alert triggering.

`analytics.py`

Returns complaint velocity grouped by state/date and model metadata such as ROC-AUC, Precision@10, training time, and feature columns.

`atms.py`

Lists ATMs with state/district filters and pagination, returns one ATM, its current risk, risk history, and an explanation of contributing factors.

`auth.py`

Registers users, logs in at `/api/v1/auth/login` and the compatibility `/auth/login`, and returns the current authenticated profile.

`complaints.py`

Creates, lists, reads, and updates complaint records. Complaint data is the primary signal used by features and spike detection.

`demo.py`

Provides demo-oriented actions such as triggering a high-risk alert. It is used to demonstrate the response workflow quickly.

`heatmap.py`

Returns GeoJSON features for ATM risk visualization at `/heatmap`. It maps numeric scores to low/medium/high severity and includes stale and cross-state flags.

`investigations.py`

Creates and lists investigations attached to complaints, reads an investigation, and records investigation events. Role dependencies restrict who may create or inspect operational work.

`mule_accounts.py`

Lists and reads mule accounts and returns linked information used to understand account geography and ATM associations.

`network.py`

Exposes a generic entity-network endpoint. It delegates graph construction to `network_service.py`.

`notifications.py`

Provides authenticated test endpoints for email and WhatsApp notification channels.

`predict.py`

Compatibility endpoint `/predict`. It returns ranked ATM predictions, optionally filters by state, and can refresh scores using the ML pipeline.

`predictions.py`

CRUD-style API for stored prediction records under `/api/v1/predictions`, with role checks for creation and filtering for reads.

`reports.py`

Admin-only `/reports/export` endpoint. It joins predictions to ATM locations, adds stale/cross-state flags, sorts by risk, and returns a downloadable CSV.

`simulation.py`

Admin simulation controls: inspect current mode, inject staged complaint spikes, and change stream mode/rate for demonstrations.

`transactions.py`

Creates, lists, and reads transaction records linked to complaints, mule accounts, and ATMs. Suspicious status and reasons support investigation context.

9. Backend setup and migration files

`backend/requirements.txt`

Python dependency list for FastAPI, SQLAlchemy, database drivers, Pydantic settings, JWT/password tooling, pandas/numpy, ML libraries, HTTP requests, and testing.

`backend/Dockerfile`

Builds the API image, installs requirements, copies backend code, and defines the container command/environment used by Compose.

`backend/Dockerfile.ml`

Alternative ML-focused image definition for environments that need a dedicated model/training container.

`backend/entrypoint.sh`

Container boot script. It waits for schema readiness through the init script, loads ATM/data/model state, and finally replaces the shell with Uvicorn.

`backend/alembic.ini`

Alembic configuration: migration script location, logging, and database migration settings.

`backend/alembic/env.py`

Connects Alembic to project settings and SQLAlchemy metadata so migration commands know the target database/schema.

`backend/alembic/versions/20260920_initial_schema.py`

Initial migration revision that creates the relational schema for the application. It is the reproducible migration counterpart to direct `Base.metadata.create_all()` development setup.

10. Backend data scripts

`backend/scripts/init_db.py`

Creates all SQLAlchemy tables with retry logic. This is useful when PostgreSQL is still starting inside Docker.

`backend/scripts/fetch_atms.py`

Attempts to fetch real ATM nodes from OpenStreetMap Overpass. If unavailable or insufficient, it generates deterministic synthetic ATM points around known Indian district centers and caches them in `data/atm_cache.json`.

`backend/scripts/generate.py`

Creates calibrated synthetic mule accounts, complaints, ATM risk history, and demo spike data. It supports batch/live/spike modes and avoids regenerating large existing datasets unless forced.

`backend/scripts/train_model.py`

Builds a labeled training dataset from current ATM features, trains/evaluates the model, saves the Joblib artifact, and populates the predictions table.

11. Frontend files

`frontend/index.html`

Vite HTML shell. It supplies the root DOM element into which React mounts.

`frontend/package.json`

Defines Vite commands and browser dependencies: React, React Router, Leaflet/react-leaflet, and Lucide icons.

`frontend/vite.config.js`

Configures Vite development/build behavior and the backend proxy so browser calls such as `/auth/login` reach FastAPI during local development.

`frontend/Dockerfile`

Builds the React application and packages the static output into an Nginx image.

`frontend/nginx.conf`

Serves the SPA static files and routes frontend requests correctly for client-side navigation. It also proxies or exposes the built application according to the container setup.

`frontend/src/main.jsx`

Browser entry point. It mounts `<App />` in React Strict Mode and loads global CSS.

`frontend/src/App.jsx`

Defines the application shell, router, authenticated navbar, login route, LEA route, Admin route, and root role-based redirect.

`frontend/src/index.css`

Global dark dashboard theme, imported fonts, scrollbar styling, Leaflet popup/map styling, and high-risk marker pulse animation.

`frontend/src/context/AuthContext.jsx`

Stores the JWT and user role in memory, performs login/logout, exposes authentication state, and creates the Authorization header for API calls.

`frontend/src/components/ProtectedRoute.jsx`

Route guard. Redirects unauthenticated users to login and sends users without an allowed role to the appropriate dashboard.

`frontend/src/components/Navbar.jsx`

Authenticated top navigation. It displays product identity, role-appropriate links, current user, and logout behavior.

`frontend/src/components/RiskMap.jsx`

Leaflet map for ATM risk. It creates colored markers from score thresholds, shows ATM details in popups, flags stale/cross-state records, and optionally draws cross-state flow lines.

`frontend/src/components/AlertFeed.jsx`

Renders active alerts with severity color, district, timestamp, message, complaint context, and cross-state indicator.

`frontend/src/pages/Login.jsx`

Login screen. It collects credentials, calls the auth context, and navigates users to the dashboard selected by their role.

`frontend/src/pages/LeaView.jsx`

LEA operational view. It periodically loads ranked predictions and alerts, filters/searches ATMs, counts high-risk locations, displays the risk map and alert feed, and offers a local patrol-deployment action.

`frontend/src/pages/AdminView.jsx`

I4C/Admin view. It loads predictions, complaint velocity, and model metrics; dispatches manual alerts; injects spike scenarios; changes simulation mode; displays intelligence panels/flows; and downloads the predictions CSV.

12. Data and documentation files

`data/atm_cache.json`

Cached ATM records produced by the ATM ingestion script. It allows repeatable startup without querying Overpass every time.

[docs/PRD.md](#)

Product requirements: problem, goals, personas, user stories, functional requirements, and success metrics.

[docs/literature_review.md](#)

Research background and evidence supporting the problem framing and design assumptions.

[docs/backend-frontend-handoff.md](#)

API contract for frontend developers: authentication, endpoint shapes, risk thresholds, alert actions, analytics, network data, simulation, and report export.

[docs/backend-frontend-handoff.pdf](#)

PDF rendering of the backend/frontend handoff document.

[scripts/render_backend_handoff.py](#)

Renderer used to convert the handoff Markdown into its PDF output.

[docs/project-file-guide.md](#)

This editable file-by-file/folder-by-folder explanation.

[scripts/render_project_guide.py](#)

Local ReportLab renderer for this guide. Re-run it after editing the Markdown source to refresh the PDF.

13. Tests: what each test protects

[backend/tests/conftest.py](#)

Shared pytest fixtures, including isolated database/client setup used by multiple tests.

[test_db.py](#)

ORM table creation, schema behavior, validation constraints, and database relationships.

[test_auth.py](#)

Login behavior, JWT claims, demo users, and role-based access.

[test_fetch_atms.py](#)

ATM fallback/import behavior and coordinate/idempotency expectations.

[test_generate.py](#)

Synthetic data counts, calibrated crime-type ratios, cross-state data, and spike generation.

test_ml.py

Feature calculations, model training, fallback behavior, evaluation metrics, and prediction output.

test_predict_heatmap.py

/predict ranking/filters and /heatmap GeoJSON shape/severity output.

test_alerts_spike.py

Spike threshold, cross-state severity escalation, alert persistence, email deduplication, and alert actions.

test_analytics_reports.py

Velocity aggregation, model metrics, and CSV report export.

test_simulation.py

Two-stage demo spike injection and simulation mode configuration.

14. Important practical notes

- Predictions are risk estimates, not proof of a crime; the UI should be used for prioritization and investigation.
- SQLite is the default local database. Docker Compose uses PostgreSQL.
- The trained model artifact is generated at runtime and is not a source file.
- `__pycache__` directories are Python-generated bytecode caches and have no business logic.
- API role names are broader than the two dashboard views: backend supports ADMIN, CITIZEN, LEA, BANK, and I4C, while the current frontend primarily routes `lea` and `admin` roles.
- The frontend keeps the JWT in memory, so a browser refresh requires logging in again by design.

15. Fast navigation map

Need to change...

API endpoint behavior
Database fields/relationships
Request or response JSON
Authentication or permissions
Risk score inputs
Model training/inference
Velocity spike behavior
Notification delivery
LEA screen
Admin screen
Map markers/popups
Login/session state
Database/data reset or seed
Container startup

Start with...

backend/app/routers/<endpoint>.py
backend/app/models.py
backend/app/schemas.py
backend/app/auth.py
backend/app/ml/features.py
backend/app/ml/model.py
backend/app/ml/spike_detector.py
backend/app/services/notifications.py
frontend/src/pages/LeaView.jsx
frontend/src/pages/AdminView.jsx
frontend/src/components/RiskMap.jsx
frontend/src/context/AuthContext.jsx
backend/scripts/
docker-compose.yml + backend/entrypoint.sh